

Audio Synchronizer Engine

High Level Design Document

*Michael **David**, Gonzales*

*Computer Science Master of Science (CSMS) Degree Program / Artificial Intelligence Graduate
Certificate*

Full Sail University

October 4, 2026

Table of Contents

Audio Synchronizer Engine	1
INTRODUCTION.....	3
Project Concept / Pitch	3
Technical Design Goal(s)	3
Development APIs / Programming Languages.....	3
Development Standards / Cold-Start Procedures.....	4
PROJECT FUNCTIONALITY	5
User Modes.....	5
Main Mechanics / Sub Mechanics	5
AI Module(s).....	6
Graphics / Animation / Video	9
Audio	9
Custom Topics	9
Additional Technical Information.....	9
Mockups / Flowcharts.....	10
PRODUCTION DOCUMENTATION	11
Master Task List.....	12
Development Roadmap	14
VERIFICATION AND VALIDATION (V&V)	15
REFERENCES.....	19

INTRODUCTION

Project Concept / Pitch

An automated Python video editor benchmarking a trained machine learning audio classifier against a heuristic beat-matching synchronization engine. (18 words)

Technical Design Goal(s)

The main technical goal is to build a clean, modular desktop script in Python that processes music tracks and raw video clips without causing the computer to freeze up or run out of memory. Since I already built the working foundation for this tool in Advanced Software Engineering (COS550), the real engineering challenge for this Capstone is setting up a head-to-head benchmark between two different ways of reading audio:

1. Heuristic Signal Processing Engine (Baseline): My original rule-based Digital Signal Processing (DSP) algorithm that calculates a sliding 100ms moving-average volume threshold to spot where the musical beats drop.
2. Trained Machine Learning (ML) Audio Classifier (Challenger): A two-step feature extraction and supervised classification pipeline built around the recent audio classification methods of Nengsih et al. [5] and Saky et al. [6]. Instead of just looking for loud volume spikes, this pipeline chops the song into short time windows using the Short-Time Fourier Transform (STFT), pulls out Musical Surface features (spectral flux, spectral centroid, spectral rolloff, zero-crossings, and Root Mean Square [RMS] energy) along with 13 Mel-Frequency Cepstral Coefficients (MFCC), and uses a trained Support Vector Machine (SVM) and k-Nearest Neighbor (kNN) model to tell actual rhythmic beat cuts apart from background noise or long vocal swells.

Under the hood, the program keeps the user file-selection workflow separated from the heavy math and model inference. That way, if a user accidentally loads a broken video file or the model hits a runtime hiccup, the script catches it, prints a clear error code, and stops safely instead of crashing the whole app.

Development APIs / Programming Languages

- Core Language: Python 3.11 - Selected for its stability, readable syntax, and native support for machine learning, audio signal processing, and video editing packages.
- Audio Signal & Feature Extraction Library: Librosa (v0.10.1) - An open-source Python tool used to read music waveforms, compute onset envelopes for the heuristic engine, and apply STFT windowing to extract Musical Surface features and MFCC vectors for the supervised classifier [1], [5], [6].

- Machine Learning Framework: Scikit-Learn & Joblib - Utilized to execute 10-fold cross-validation and 70/30 train/test splits, feature scaling, supervised model training (SVM with polynomial, Radial Basis Function [RBF], and sigmoid kernels, benchmarked against kNN), confusion matrix generation, and model serialization (.pkl) [5], [6].
- Video Slicing & Compositing Library: MoviePy (v1.0.3) & FFmpeg - A video toolkit deployed to slice video files apart at specific timestamps, stitch them back together in chronological order, and render a final H.264 .mp4 video file to the desktop [2].
- System Operations & Unit Testing: Python's built-in os, sys, and time modules used to read local computer file paths, check directory folders, measure audio detection time in milliseconds, and print console status telemetry [3], paired with pytest for automated unit verification.

Development Standards / Cold-Start Procedures

To ensure this software can run cleanly on any new laptop or workstation with minimal assistance, the project enforces these development standards and setup procedures

GitHub Repository Link: [//github.com/michaeldavid90069-AiEngineer/COS660-michaeldavid90069-AiEngineer](https://github.com/michaeldavid90069-AiEngineer/COS660-michaeldavid90069-AiEngineer)

- Integrated Development Environment (IDE) & Versions: Visual Studio Code (v1.85+) with the Python extension, running Python 3.11.0 and Git (v2.40+).
- Coding & Engineering Principles: Source code follows Python Enhancement Proposal 8 (PEP 8) formatting conventions, uses explicit type hints, separates logic into modular classes (AudioOrchestrator, SignalProcessor, and MLAudioClassifier), and excludes virtual environments and temporary media files via a root Python .gitignore file.
- Folder Architecture:
 - main.py (The primary script that runs the user interface and pipeline controller).
 - /src (The folder containing modular backend scripts: heuristic_engine.py, ml_classifier.py, and video_compiler.py).
 - /models (The folder storing serialized trained classifier weights audio_classifier.pkl and feature scalers).
 - /assets (The folder where the user drops unedited .mp4 videos and the background .mp3 or .wav music file).

- /output (The folder where the finished, rhythm-matched video files output_heuristic.mp4, output_ai.mp4, or anonymized Sample_A.mp4 and Sample_B.mp4 test pairs are saved).
- Step-by-Step Cold-Start Setup Order:
 1. Clone the repository to the local computer: git clone [//github.com/michaeldavid90069-AiEngineer/COS660-michaeldavid90069-AiEngineer](https://github.com/michaeldavid90069-AiEngineer/COS660-michaeldavid90069-AiEngineer)
 2. Open a terminal inside the project folder and initialize a local virtual environment to avoid library version conflicts: python -m venv venv
 3. Activate the virtual environment (venv\Scripts\activate on Windows or source venv/bin/activate on macOS/Linux).
 4. Install all locked dependencies from the plain-text requirements.txt file: pip install -r requirements.txt
 5. Place unedited .mp4 video clips and an .mp3 or .wav music file inside the /assets folder.
 6. Run python main.py in the terminal to execute the pipeline and save the synchronized .mp4 video inside /output.

PROJECT FUNCTIONALITY

User Modes

- Single User / Creator Mode: The program functions entirely on a local desktop machine. The individual user retains control over picking the input files, pacing settings (High or Low), and active processing engine (Heuristic or AI_Classifier), while the program calculates the audio beat coordinates and cuts the footage.
- Blind A/B Evaluation Sub-Mode (User Study Mode): A preconfigured evaluation sub-mode designed for within-subjects user testing. The program processes the same source video clips and music track through both the Heuristic Engine and the Trained AI Classifier, exporting anonymized video files (Sample_A.mp4 and Sample_B.mp4) so human testers can blindly evaluate one output against the other.

Main Mechanics / Sub Mechanics

- File Selection & Header Validation: The system provides an asset-loading interface where a user selects their video tracks (.mp4) and a music file (.mp3 or .wav), executing try-except exception blocks to verify binary file headers before loading.

- **Rhythm Tracking & Supervised Audio Classification:** The script feeds the music file into either the baseline moving-average algorithm (which searches for rapid spikes in volume to map out the music's downbeats) or the supervised machine learning classifier (which extracts STFT Musical Surface and MFCC features to classify beat cut frames) [5], [6].
- **Footage Slicing:** The system loops through the unedited videos and slices them into smaller sub-clips that fit within the time gaps between the extracted music beat timestamps.
- **Timeline Assembly:** The program chains the edited video pieces back together chronologically, attaches the stereo song underneath, and writes it out as a brand-new .mp4 file.
- **Researched Techniques, Backup Solutions & Published Examples:** Short-time audio feature extraction followed by supervised classification is the established standard for music information retrieval and audio event classification [5], [6]. If the trained machine learning model encounters an unusual audio file that produces low prediction confidence across all frames, the system automatically falls back to the deterministic COS550 moving-average heuristic algorithm so the user always receives a completed video file. Published commercial products featuring comparable beat-matching mechanics include CapCut (Auto Beat Sync) and Adobe Premiere Pro.

AI Module(s)

To meet the graduate Artificial Intelligence requirement and test whether a trained model actually beats a rule-based script in practice, the app runs two competing modules side by side:

1. **Baseline Heuristic Module (SignalProcessor):** Instead of looking for one fixed volume number to spot a beat (which completely fails when a song has a quiet intro and a loud chorus), the baseline code calculates a rolling 100-millisecond moving-average volume threshold as the song plays.
2. **Trained Supervised Audio Classifier (MLAudioClassifier):** The catch with only tracking volume is that a loud vocal note or a guitar swell can still trick a basic threshold into making a bad video cut. To fix that, the AI module uses the two-step feature extraction and supervised classification approach outlined in recent studies by Nengsih et al. [5] and Saky et al. [6].
 - **Short-Time Feature Extraction:** Running a standard Fast Fourier Transform (FFT) on a whole song tells you what frequencies are in the track, but it loses track of *when* they happen. Because music is constantly changing over time, the audio wave is sliced into narrow overlapping time frames using the Short-Time Fourier Transform

(STFT). For every short frame, Librosa [1] pulls an 18-number feature list made up of 5 Musical Surface descriptors (spectral flux, spectral centroid, spectral rolloff, zero-crossings, and RMS energy) plus 13 Mel-Frequency Cepstral Coefficients (MFCC) that capture the actual texture and timbre of the sound. Frame volume via RMS energy is calculated using the standard plain-text formula. $RMS = \sqrt{(\sum_{n=1}^N x_n^2) / N}$

where N is the number of audio samples in that frame window and x_n^2 is the squared amplitude at sample n . To map frequencies the way human ears actually hear pitch, the Mel-scale filter bank is calculated using the plain-text formula:

$$F_{mn} = 2595 * \log_{10}(1 + (f / 700))$$

where f is the frequency (starting at 1 kHz), giving the classifier local acoustic features that hold up even when the pitch or tempo shifts.

- Supervised Classification & Cross-Validation: Once those 18 features are extracted for each frame, they are fed into a supervised Support Vector Machine (SVM) and compared against a k-Nearest Neighbor (kNN) benchmark. Since audio data rarely separates in a straight line, the SVM tests three non-linear kernel functions to find the cleanest boundary between beat frames and non-beat frames:
 - Polynomial Kernel: $K(X_i, X_j) = (\gamma * X_i * X_j + C)^d$
 - Radial Basis Function (RBF) Kernel: $K(X_i, X_j) = \exp(-\gamma * |X_i - X_j|^2)$
 - Sigmoid Kernel: $K(X_i, X_j) = \tanh(\gamma * X_i * X_j + C)$

Following the evaluation setup used in [5] and [6], the classifier is tested using both a 70%/30% train/test split and 10-fold cross-validation with a confusion matrix so we know the numbers aren't just a fluke on one song.

- Performance Metrics & Risk Mitigation: To keep the comparison objective in Python, the script logs four core metrics across every test run:
 1. Audio Detection Time (AD_time in milliseconds):
 $AD_time = \sum_{i=1}^n (S_i * Time[AD])$
 tracking how long the computer takes to process the audio samples S_i .
 2. Audio Detection Accuracy (AD_acc in percentage %):
 $AD_acc = \sum_{i=1}^n (S_{ADA} / S_i)$

tracking the percentage of audio frames S_ADA classified correctly out of all frames S_i .

3. Precision Rate (P):

$$P = TP / (TP + FP)$$

tracking True Positives (TP) against False Positives (FP) so the video doesn't get chopped up on false beats.

4. Recall Rate (R):

$$R = TP / (TP + FN)$$

tracking True Positives (TP) against False Negatives (FN) so real downbeats aren't missed.

To keep the CPU from bogging down during feature extraction (AD_time), stereo songs are flattened to a single mono channel during the analysis step, and a 10% time buffer is built into every milestone in case the model needs extra tuning.

ALGORITHM: AudioBeatPacingOrchestrator

INPUT: MusicFile (mp3/wav), RawVideoAssets (Array of mp4), SpeedFlag (High/Low),

EngineMode (Heuristic/AI_Classifier/AB_Compare)

OUTPUT: SynchronizedTimelineArray, EvaluationTelemetry

```
BEGIN
// Step 1: Ingest and flatten signal vector
AudioSignal, SampleRate = Call librosa.load(MusicFile, mono=True)

// Step 2A: Compute dynamic moving-average threshold window (Heuristic Baseline)
IF EngineMode == "Heuristic" OR EngineMode == "AB_Compare" THEN
  OnsetEnvelopes = ComputeVolumeTransientSlopes(AudioSignal)
  DynamicThreshold = CalculateMovingAverage(OnsetEnvelopes, window_size=100ms)
  HeuristicTimestamps = []
  FOR EACH Index, EnvelopeValue IN OnsetEnvelopes DO
    IF EnvelopeValue > DynamicThreshold THEN
      Append (Index / SampleRate) TO HeuristicTimestamps
    END IF
  END FOR
END IF

// Step 2B: STFT Musical Surface & MFCC Supervised Classification (Trained AI Module)
IF EngineMode == "AI_Classifier" OR EngineMode == "AB_Compare" THEN
  TrainedSVMModel, Scaler = Call Joblib.load("models/audio_classifier.pkl")
  AudioFrames = ApplySTFTWindowing(AudioSignal, frame_length=50ms, hop_length=25ms)
  AllTimestamps = []
  FOR EACH FrameIndex, FrameSignal IN AudioFrames DO
    MusicalSurface = ExtractMusicalSurface(FrameSignal, Flux, Centroid, Rolloff, ZCR, RMS)
    MFCCVector = ExtractMFCC(FrameSignal, n_mfcc=13)
    FeatureVector = Concatenate(MusicalSurface, MFCCVector)
    NormalizedVector = Scaler.transform(FeatureVector)
    PredictionClass = TrainedSVMModel.predict(NormalizedVector)
    IF PredictionClass == 1 THEN
      Append (FrameIndex * 0.025) TO AllTimestamps
    END IF
  END FOR
END IF

// Step 3: Apply user pacing density filter logic and slice timeline
FOR EACH TimestampArray IN SelectedEnginePipelines DO
  IF SpeedFlag == "Low" THEN
    TimestampArray = FilterAlternatingElements(TimestampArray)
  END IF

  // Step 4: Timeline clipping loop
  TimelineSegments = []
  FOR I FROM 0 TO Length(TimestampArray) - 2 DO
    StartTime = TimestampArray[I]
    EndTime = TimestampArray[I+1]
    VideoClip = SelectAssetSource(RawVideoAssets)
    CutSegment = Call ffmpeg_extract_subclip(VideoClip, StartTime, EndTime)
    Append CutSegment TO TimelineSegments
  END FOR

  // Step 5: Multi-track composite compilation
  CompiledVideoTrack = ConcatenateTimeline(TimelineSegments)
  FinalMuxedOutput = MixAudioVideoTracks(CompiledVideoTrack, MusicFile)
  WriteVideoFile(FinalMuxedOutput, OutputFolder)
END FOR

RETURN FinalMuxedOutput, EvaluationTelemetry
END
```

User Interface / Inputs / Sensors

- **File Path & Mode Controls:** Straightforward navigation buttons and configuration parameters allowing the user to select local folders, specify pacing settings (High to cut on every beat or Low to skip alternating beats), and select the processing mode (Heuristic, AI_Classifier, or AB_Compare).
- **Status Telemetry:** Clear text printed directly to the screen (sys.stdout) telling the user what the program is doing in real time ("Analyzing Audio Tracks... 45%", audio detection time AD_time in milliseconds, or "Rendering Final Video"). No specialized hardware sensors or external peripherals are required.

Graphics / Animation / Video

- **Target Codec:** The software exports standard .mp4 video files using the H.264 format via MoviePy and FFmpeg loaders/exporters, ensuring the finished clip plays properly across desktop media players, social media, and phone apps.
- **Resolution:** The tool preserves the original 1080p high-definition layout and frame rates of the imported video clips without requiring custom 3D rendering or shaders.

Audio

- **Sound Standard:** The engine loads standard 44.1 kHz (as well as 22.05 kHz and 16 kHz benchmark) .mp3 and .wav audio files.
- To save processing power during heuristic peak-picking and STFT/MFCC feature extraction, it temporarily flattens multi-channel stereo audio into a single mono channel to calculate the beats, then binds the full-quality stereo sound back onto the final video clip when rendering. No voice acting assets are required.

Custom Topics

- **Within-Subjects A/B User Evaluation Protocol:** In alignment with faculty advisor guidance, the software incorporates a side-by-side experimental test bundler. When executed in AB_Compare mode, the system generates two anonymized video exports—one cut by the COS550 heuristic moving-average baseline and one cut by the trained SVM/MFCC audio classifier—so human test participants can evaluate both videos in a blind within-subjects study and rate rhythmic synchronization and transition flow.

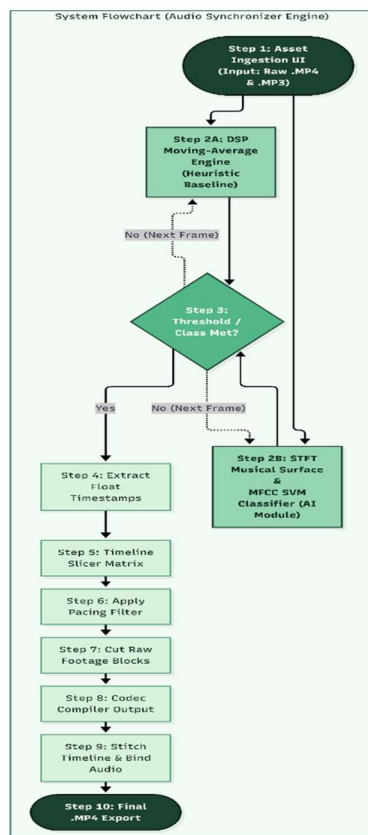
Additional Technical Information

The following potential problems, bottlenecks, and edge cases have been researched along with practical fixes:

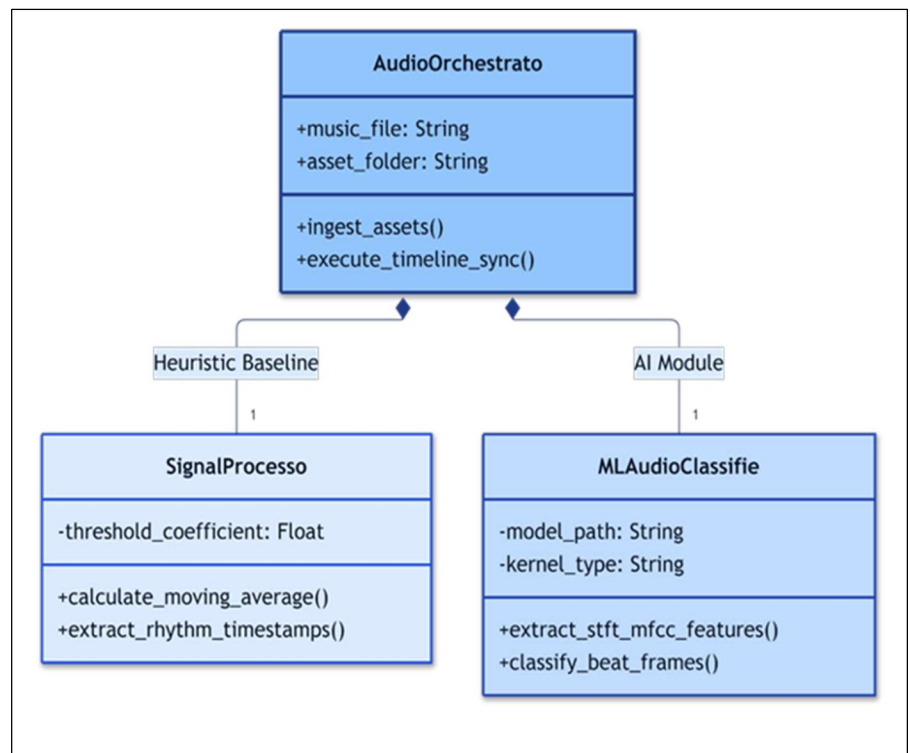
- **High Dimensionality & Noise in Audio Event Classification:** As shown in the musical instrument classification work by Nengsih et al. [5] and the multi-modal acoustic classification study by Saky et al. [6], raw audio files are packed with data, and background noise or overlapping instruments can easily confuse a classifier and slow down processing. Solution: Instead of feeding raw waveforms into the model, compress each audio frame down to an 18-number STFT feature list (5 Musical Surface descriptors + 13 MFCCs) and validate the model using 10-fold cross-validation across non-linear SVM kernels and kNN baselines [5], [6].
- **Workstation Memory Exhaustion During Rendering:** Loading dozens of 1080p video clips into memory all at once in MoviePy will quickly eat up RAM and freeze the machine. Solution: Cut sub-clips directly from the file stream using `ffmpeg_extract_subclip` and close out file handles right after stitching the timeline together.
- **Rapid False-Positive Micro-Cuts:** Fast drum rolls can trigger beat timestamps less than 100 milliseconds apart, which makes the video look like a strobe light. Solution: Put a 250-millisecond cooldown timer between back-to-back cuts, and give the user a Low pacing setting that skips every other beat.

Mockups / Flowcharts

System Flowchart (Audio Synchronizer Engine)



Unified Modeling Language (UML) Class Diagram



PRODUCTION DOCUMENTATION

Feature List

Number	Feature	Category	Priority	Difficulty
F1	Local Asset Ingestion System – Safe file Input/Output (I/O) streams scanning local asset directories [3].	System	High	Easy
F2	Corrupted & Format File Validation – Native Python exception handlers verifying .mp4, .mp3, and .wav file headers [3].	System	High	Medium
F3	Heuristic Audio Beat Analysis Core (Baseline) – Librosa moving-average transient slope detector [1], [4].	System	High	Medium
F4	STFT Musical Surface & MFCC Feature Extractor – Short-time frame extractor computing spectral flux, spectral centroid, spectral rolloff, zero-crossings, RMS energy, and 13 MFCCs [1], [5], [6].	System	High	Hard
F5	Supervised ML Audio Classifier Module (Challenger) – Trained Scikit-Learn SVM and kNN classification pipeline outputting beat predictions and accuracy, precision, recall, and detection time metrics [5], [6].	System	High	Hard
F6	Pacing Density & Engine Mode Selector – User control parameters modifying clip frequency (High/Low) and active pipeline (Heuristic, AI_Classifier, or AB_Compare) [3].	UI	Normal	Easy
F7	Synchronized Media Slicing Renderer – MoviePy and FFmpeg timeline engine slicing video segments at timestamp coordinates [2].	System	High	Hard
F8	Master Multi-Track Audio/Video Export & A/B Bundler – MoviePy file compiler muxing stereo audio and video streams into .mp4 containers for single or blind A/B output [2].	System	High	Medium

F9	Live Console Progress & Metrics Telemetry – Standard Python sys.stdout stream printing live execution percentages and classifier evaluation metrics [3], [5].	UI	Low	Easy
F10	Python 3.11, Librosa, Scikit-Learn, MoviePy & Git – Core production tools and libraries used to implement and version-control the software [1]–[6].	Tool	High	Medium

Category:

- UI: features directly impacting the user
- System: system to support feature content
- Tools: production tools used to implement the feature

Master Task List

Number	Task	Feature	Dependencies
T1	Run python -m venv venv, configure the repository .gitignore, and write dependency package versions (librosa, scikit-learn, moviepy, pytest) to a requirements.txt configuration file according to standard Python packaging guidelines [3].	F1	NA
T2	Parse local folder arguments using the native Python os.path module [3] to return absolute file paths from the computer disk.	F1	T1
T3	Execute try-except exception catch blocks to evaluate and read binary header flags of .mp4, .mp3, and .wav files as defined by standard file format specifications [3].	F2	T2
T4	Call the librosa.load function to ingest audio tracks and output a single-channel floating-point data array mapping raw signal voltage values [1].	F3, F4	T3
T5	Compute local volume transient shifts for the heuristic baseline by executing a 100 ms moving-average mathematical threshold loop across the audio array using Librosa's onset detection parameters [1], [4].	F3	T4
T6	Export an array index list containing the exact float timestamps of every calculated heuristic rhythmic peak based on the digital signal processing grid [4].	F3	T5

T7	Segment the mono audio array into short-time frames via STFT in Librosa [1] and extract an 18-feature vector per frame consisting of 5 Musical Surface features (spectral flux, spectral centroid, spectral rolloff, zero-crossings, and RMS energy where $RMS = \sqrt{(\sum_{n=1}^N x_n^2) / N}$) and 13 MFCCs [5], [6].	F4	T4
T8	Train and evaluate supervised Support Vector Machine (comparing polynomial, RBF, and sigmoid kernels) and kNN classifiers in Scikit-Learn using a 70/30 train/test split and 10-fold cross-validation, generating a confusion matrix along with accuracy ($AD_acc = \sum(S_ADA / S_i)$), precision ($P = TP / (TP + FP)$), and recall ($R = TP / (TP + FN)$) metrics [5], [6].	F5	T7
T9	Serialize the best-performing trained classifier to /models/audio_classifier.pkl and execute runtime inference on input tracks to export AI-predicted beat timestamp arrays while logging audio detection time ($AD_time = \sum(S_i * Time[AD])$ in ms) [5], [6].	F5	T8
T10	Evaluate user choice configuration parameters via conditional statements to prune or retain alternating timestamp array elements (High/Low pacing) and select the active pipeline (Heuristic, AI_Classifier, or AB_Compare) [3].	F6	T6, T9
T11	Call moviepy.video.io.ffmpeg_tools.ffmpeg_extract_subclip [2] to programmatically split source video files at the exact timestamp array indices.	F7	T3, T10
T12	Concatenate the individual sliced video elements into a continuous timeline tracking array using MoviePy's sequence array compilation rules [2] and release closed file handles.	F7	T11
T13	Combine the original background stereo audio track stream layer with the compiled chronological video timeline array layer via MoviePy multi-track composition hooks [2].	F8	T12
T14	Call the MoviePy.write_videofile() system compiler [2] to render H.264 encoded .mp4 file containers to the local /output workspace folder, generating anonymized Sample_A.mp4 and Sample_B.mp4 pairs when run in A/B comparison mode.	F8	T13
T15	Print ongoing calculations, progress percentages (0% to 100%), and classifier evaluation metrics (AD_time,	F9	T9, T14

	AD_acc, P, R) to the system's standard output (sys.stdout) stream [3], [5] by dividing processed frame indices by total timeline frame counts.		
T16	Conduct a blind within-subjects A/B user evaluation comparing Sample_A.mp4 (Heuristic) and Sample_B.mp4 (Trained AI Classifier) across test tracks and record participant preference and sync ratings [6].	F5, F8	T14, T15

Development Roadmap

Milestone 1 (Estimated: 12 Hours + 1.2 Hours [10% Padding] = 13.2 Hours)

- T1 - Run `python -m venv venv`, configure the repository `.gitignore`, and write dependency package versions (librosa, scikit-learn, moviepy, pytest) to a `requirements.txt` configuration file according to standard Python packaging guidelines [3].
- T2 - Parse local folder arguments using the native Python `os.path` module [3] to scan `/assets` and return absolute file paths from the computer disk.
- T3 - Execute try-except exception catch blocks to evaluate and read binary header flags of `.mp4`, `.mp3`, and `.wav` files as defined by standard file format specifications [3], halting safely if corrupted media is detected.
- T4 - Call the `librosa.load` function to ingest audio tracks and output a single-channel floating-point data array mapping raw signal voltage values [1].
- T5 - Compute local volume transient shifts for the heuristic baseline by executing a 100ms moving-average mathematical threshold loop across the audio array using Librosa's onset detection parameters [1], [4].
- T6 - Export an array index list containing the exact float timestamps of every calculated heuristic rhythmic peak based on the digital signal processing grid [4].

Milestone 2 (Estimated: 16 Hours + 1.6 Hours [10% Padding] = 17.6 Hours)

- T7 - Segment the ingested mono audio signal into short-time frames using STFT windowing in Librosa [1] and compute an 18-dimensional acoustic feature vector per frame comprising 5 Musical Surface features (spectral flux, spectral centroid, spectral rolloff, zero-crossings, and RMS energy where $RMS = \sqrt{(\text{sum from } n=1 \text{ to } N \text{ of } x_n^2) / N}$) and 13 Mel-Frequency Cepstral Coefficients (where $F_{mn} = 2595 * \log_{10}(1 + (f / 700))$) [5], [6].

- T8 - Train and benchmark supervised Support Vector Machine (testing polynomial, RBF, and sigmoid kernels) and kNN classifiers using both a 70%/30% train/test split and 10-fold cross-validation, outputting a confusion matrix and calculating audio detection accuracy ($AD_acc = \sum(S_ADA / S_i)$), precision ($P = TP / (TP + FP)$), and recall ($R = TP / (TP + FN)$) [5], [6]. (Includes a 10% schedule buffer for kernel hyperparameter tuning).
- T9 - Serialize the highest-accuracy trained classifier to /models/audio_classifier.pkl and implement the runtime inference loop to export AI-classified beat timestamp arrays while measuring audio detection time ($AD_time = \sum(S_i * Time[AD])$ in ms) [5], [6].
- T10 - Evaluate user choice configuration parameters via conditional statements to prune or retain alternating timestamp array elements (High vs. Low pacing) and route execution through Heuristic, AI_Classifier, or AB_Compare modes [3].

Milestone 3 (Estimated: 16 Hours + 1.6 Hours [10% Padding] = 17.6 Hours)

- T11 - Call moviepy.video.io.ffmpeg_tools.ffmpeg_extract_subclip [2] to programmatically split source video files at the exact timestamp array indices generated by the active pipeline(s).
- T12 - Concatenate the individual sliced video elements into a continuous timeline tracking array using MoviePy's sequence array compilation rules [2] and close temporary file handles.
- T13 - Combine the original background stereo audio track stream layer with the compiled chronological video timeline array layer via MoviePy multi-track composition hooks [2].
- T14 - Call the MoviePy.write_videofile() system compiler [2] to render 1080p H.264 encoded .mp4 file containers to the local /output workspace folder, generating anonymized Sample_A.mp4 (Heuristic) and Sample_B.mp4 (Trained AI Classifier) video exports when run in A/B comparison mode.
- T15 - Print ongoing calculations, progress percentages (0% to 100%), audio detection time (AD_time), and classification telemetry to the system's standard output (sys.stdout) stream [3], [5] by dividing processed frame indices by total timeline frame counts.
- T16 - Execute a blind within-subjects A/B user evaluation study comparing the Heuristic baseline video (Sample_A.mp4) against the Trained AI Classifier video (Sample_B.mp4) and compile comparative user ratings [6].

VERIFICATION AND VALIDATION (V&V)

Quality Test List

Task	Verification	Validation
T1	The terminal screen displays virtual environment activation confirmation, and pip list verifies all packages in requirements.txt are installed with zero version conflict errors [3].	The developer possesses an isolated, sandboxed Python ecosystem containing all required external multimedia and machine learning libraries.
T2	The console prints a structured string list containing the absolute system paths of 100% of .mp4, .mp3, and .wav files discovered inside /assets via the os.path module [3].	The application points to and locates raw editing assets saved inside the local workspace folder.
T3	Passing an uncorrupted media file raises zero header exceptions, while passing a zero-byte or corrupted file triggers the try-except block and prints an error code within 500 ms without crashing the process [3].	The video editor loads valid files safely without corrupt media breaking the application loop.
T4	An automated pytest unit check verifies that a non-empty, single-channel 1D floating-point numpy array is populated in system memory via the librosa.load function [1].	The background music is converted into a flat mono waveform format that both the heuristic and AI modules can read.
T5	On a 120 Beats Per Minute (BPM) reference audio track, the heuristic moving-average loop outputs mathematical transient data indices within +/- 25 ms whenever an amplitude slope crosses the dynamic threshold using Librosa's onset parameters [1], [4].	The baseline heuristic program traces changes in a musical rhythm to identify where the core downbeats drop.
T6	The heuristic module exports a strictly ascending sequential list of timeline	The baseline system establishes a chronological

	float coordinates snapped to exact video frame intervals (1 / FPS) based on the DSP grid [4].	blueprint layout grid for the video cuts to follow.
T7	A pytest shape assertion confirms the STFT feature extractor outputs a 2D numeric matrix of shape (N_frames, 18) (5 Musical Surface features + 13 MFCCs) containing zero NaN or infinite values [1], [5], [6].	Timbral, rhythmic, and spectral audio properties are captured across every short-time frame of the song for machine learning classification.
T8	Running the training script completes 10-fold cross-validation and outputs a confusion matrix achieving classification accuracy (AD_acc) $\geq 76.6\%$ alongside precision ($P = TP / (TP + FP)$) ≥ 0.80 and recall ($R = TP / (TP + FN)$) ≥ 0.80 on the held-out test set [5], [6].	The trained machine learning classifier distinguishes true musical beat cut points from background noise and non-percussive vocal swells.
T9	Loading /models/audio_classifier.pkl and executing inference on a 3-minute song logs an audio detection time (AD_time) under 2000 milliseconds and returns a sorted float timestamp array [5], [6].	The user generates AI-classified beat cut coordinates locally on a desktop computer in under two seconds.
T10	Conditional execution gates reduce the active timestamp array length by 50% (+/- 1 element) when SpeedFlag is set to Low versus High, and queue both engines when EngineMode is set to AB_Compare [3].	The editor can choose whether the video cuts quickly or slowly, or runs a side-by-side Heuristic vs. AI comparison, by changing a single parameter setting.
T11	External system calls to ffmpeg_extract_subclip [2] output N - 1 separate, trimmed video sub-clips saved into the temporary	Raw, unedited footage blocks are programmatically cut into smaller individual scenes at the exact rhythm markers.

	directory whose durations match the intervals between consecutive timestamps within +/- 1 frame.	
T12	The compilation matrix combines the distinct visual segments into a single continuous MoviePy timeline sequence whose duration equals the sum of all sub-clips [2].	The individual video clips flow consecutively in chronological order as a single unified sequence track.
T13	A timeline composition check verifies that the 2-channel stereo audio track layer and the H.264 video timeline layer are bound together with matching durations via MoviePy multi-track composition hooks [2].	The full-quality stereo background music layer aligns and plays alongside the moving video.
T14	H.264 encoded .mp4 file containers (Sample_A.mp4 and Sample_B.mp4 in A/B mode, each > 1 MB) are written directly to the local /output disk path using the .write_videofile() system compiler [2].	The content creator receives finalized, synchronized 1080p video files that play smoothly in native desktop media players.
T15	The active console window continuously updates progress percentages from 0% to 100% directly to the sys.stdout stream alongside AD_time, AD_acc, precision, and recall telemetry [3], [5].	The user can monitor the status of video rendering and model evaluation in real time without the program appearing frozen.
T16	A structured evaluation table logs blind within-subjects participant ratings (1-5 Likert scale for beat sync accuracy and visual transition quality) comparing Sample_A.mp4 (Heuristic) against Sample_B.mp4 (AI Classifier) [6].	Human evaluators blindly test the Heuristic engine against the Trained AI Classifier, validating which pipeline produces cleaner rhythm-synchronized video edits.

REFERENCES

- [1] Librosa Development Team, "Librosa v0.10.2 Documentation: Audio and Time-Series Signal Processing in Python," Librosa Core Modules, 2025. [Online]. Available: <https://librosa.org/doc/latest/>
- [2] MoviePy Development Team, "MoviePy v2.0 / v1.0.3: Video Editing Administration and Compositing Framework Suite," MoviePy Framework Documentation, 2025. [Online]. Available: <https://zulko.github.io/moviepy/>
- [3] Python Software Foundation, "The Python Standard Library Framework: v3.11 / v3.13 Core OS and System Output Documentation," Python Core Specifications, 2026. [Online]. Available: <https://docs.python.org/3/library/>
- [4] J. G. Proakis and D. G. Manolakis, "Onset Detection and Discrete Time Vector Processing," in *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2007, ch. 4, pp. 185-214.
- [5] W. Nengsih, A. H. K. Ching, Zulkarnain, and D. Faustine, "Deep learning-based classification of Indonesian traditional musical instruments using acoustic features," *NED University Journal of Research, Special Issue*, vol. 23, pp. 338-348, Jul. 2026. Accessed: Oct. 1, 2026. [Online]. Available: Full Sail University Library, EBSCO Academic Search Complete, <https://oclc.fullsail.edu>.
- [6] S. M. A. I. Saky, M. S. Arefin, M. R. Islam, M. S. Islam, R. I. Sumon, M. M. R. Masud, M. Lapina, M. Babenko, and M. Muthanna, "Explainable multi-modal deep learning for recording-level classification of respiratory audio signals under internal and domain-shift evaluation," *Life*, vol. 16, no. 7, p. 1108, Jul. 2026, doi: 10.3390/life16071108. Accessed: Oct. 1, 2026. [Online]. Available: Full Sail University Library, EBSCO Academic Search Complete, <https://oclc.fullsail.edu>.

APPENDIX A: ADVISOR FEEDBACK INTEGRATION NOTES

Advisor Name: Dr. Oleg Kachirski Student Name: Michael David Gonzales Consultation Dates: September 30, 2026 - October 2, 2026 (via Discord Direct Messages and Friday Zoom Weekly Advisor Check-In)

1. Advisor Feedback Provided on Previous Draft (COS550 Baseline):
 - Project Viability & Two-Month Certificate Scope: Dr. Kachirski looked over my COS550 High-Level Design Document (Audio Synchronizer Engine) and confirmed that I don't need to start from scratch—building on my existing project works well for the two-month Graduate Certificate Capstone track.
 - Adding an AI vs. Heuristic Benchmark & User Evaluation: Dr. Kachirski pointed out that my COS550 script used a rule-based heuristic (a moving-average volume cutoff) rather than an AI model. He suggested turning the project into a side-by-side comparison—pitting my heuristic engine against an AI audio processing approach—and having 10 to 15 testers compare both outputs in a within-subjects study using a 0-5 Likert scale to test our main hypothesis (that AI improves sync quality) against a backup null hypothesis (that the heuristic baseline performs just as well).
2. How Advisor Feedback Was Integrated into This Document:
 - Dual-Pipeline Setup (Introduction & AI Modules): Kept the COS550 moving-average detector as Pipeline A (Heuristic Baseline) and added Pipeline B (Trained Supervised Audio Classifier) so the app can run both head-to-head on the same song and video clips.
 - Recent 2026 Peer-Reviewed Library Research: Pulled two July 2026 peer-reviewed journal papers from the Full Sail University Library (oclc.fullsail.edu / EBSCO Academic Search Complete)—Nengsih et al. [5] and Saky et al. [6]—to back up the 18-feature STFT/MFCC extraction pipeline and the SVM/kNN cross-validation benchmarks without using unreviewed arXiv preprints.
 - Updated Task List & 10%-Padded Roadmap (T1-T16): Added features F4, F5, and F8 along with tasks T7-T9 and T16 to cover STFT/MFCC extraction, 10-fold cross-validation, saving the trained model (.pkl), and exporting blind A/B test videos (Sample_A.mp4 and Sample_B.mp4) with a 10% time buffer on every milestone.
 - Measurable V&V Targets: Added concrete developer test numbers for detection speed ($AD_time < 2000\text{ ms}$), accuracy ($AD_acc \geq 76.6\%$), precision ($P \geq 0.80$), and recall ($R \geq 0.80$), plus the blind within-subjects Likert-scale user evaluation.

APPENDIX B: AI LEVEL 3 DISCLOSURE (PROMPTS & ORIGINAL WORK)

1. Prompts Used During AI-Assisted Editing:

- "audio classifier to go head-to-head against COS550 heuristic engine"
- "okay lets implement of discussion with our advisor Dr.Oleg into the (Re)submit Design Document"
- "be sure to format it into the provided template provided over the FSO (see attachment)"
- "did you utilize the Full Sail Library From the course website (LMS); Through Full Sail One at <https://one.fullsail.edu/support/articles/657>; or, Through the Library's proxy server at <https://oclc.fullsail.edu> as Dr. Rebecca Leis informed us that its mandatory"
- "Regenerate the high-level design document with the updated changes applied from the Full Sail University library databases."
- "Regenerate with everything in plain text, including the equation."
- "generate the text for 4. Add the AI Level 3 & Advisor Feedback Appendices After Page 20"

2. Original Student-Created Work (Unedited COS550 Baseline Document - No AI Content):

Audio Synchronizer Engine High Level Design Document Michael David, Gonzales CSMS Degree Program Full Sail University 07/06/2026

Document Overview & Purpose

Engineering Objective The purpose of this High-Level Design (HLD) Document is to establish a rigorous, transparent blueprint for an automated, rhythm-driven video synchronization and orchestration application. By decoupling developmental assumptions from execution, this document codifies architecture, resource constraints, and operational dependencies into a predictable workflow. The technical specifications herein are designed to ensure that any independent software engineer can review this architecture, understand the design decisions, and execute project implementation without ambiguity.

Document Scope & Structure This blueprint establishes the foundation for building the software, organized across key engineering domains:

- **System Architecture & Mockups:** Visualizing systemic entry points and data interfaces.

- Master Task List & Schedule: Categorizing modular engineering tasks (T1 through T12) explicitly mapped to their underlying external specifications and library documentation using strict IEEE formatting standards.
- Development Roadmap: Allocating tasks across targeted milestones to ensure systematic iterative builds.
- Verification & Validation (V&V) Matrix: Outlining clear, objective criteria to confirm software compliance from both an operational developer perspective and an experiential user perspective.

Intended Audience This technical brief serves as the authoritative source of truth for:

- Academic Reviewers & Evaluators: To gauge structural completeness, validation rigor, and adherence to signal processing requirements.
- Independent Developers: Providing a step-by-step implementation roadmap tracking dependencies and package rules.
- Industry Peers & Stakeholders: Showcasing clean software design practices, estimation safety buffers, and systematic architectural patterns.

Introduction

Project Concept / Pitch A simple Python program that automatically cuts raw video clips to match the exact rhythmic beat of a background music file.

Technical Design Goal(s) The main technical goal is to build a clean, easy-to-understand desktop script using Python. The script must process music tracks and video clips smoothly without causing the computer to freeze or run out of memory. Structurally, the application will separate the user file-selection process from the heavy underlying engineering calculations. This design choice guarantees that if a video file happens to be corrupted or broken, the program will halt safely and display an error code to the user instead of crashing the entire script.

Development APIs / Programming Languages

- Core Language: Python 3.11 - chosen because it is highly stable, user-friendly, and natively supports powerful audio and video processing packages.
- Audio Library: Librosa - an open-source Python tool utilized to read music waveforms, analyze local audio data, and extract a clean list of timestamps showing where beats occur.

- Video Library: MoviePy - a video toolkit deployed to cut video files apart at specific timestamps, stitch them back together in order, and render a final video file to the desktop.
- System Operations: Python's built-in os module used to read local computer file paths, check directory folders, and save the exported file.

Development Standards / Cold-Start Procedures To ensure this software can run cleanly on any new laptop or computer, the project enforces these basic guidelines:

- Virtual Isolation: Developers must initialize a local virtual environment (`python -m venv venv`) to avoid software library version conflicts with other installed applications.
- Locked Dependencies: All required Python packages are explicitly declared inside a plain text `requirements.txt` file, allowing an engineer to install everything using the simple terminal command `pip install -r requirements.txt`.
- Folder Architecture: The directory structure is kept minimal to isolate source files from final exports:
 - `main.py` (The primary script that runs the user interface).
 - `/assets` (The folder where the user drops unedited videos and the background music file).
 - `/output` (The folder where the finished, rhythm-matched video file is saved).

Project Functionality

User Modes

- Single User / Creator Mode: The program functions entirely on a local desktop machine. The individual user retains full control over picking the input files and pacing settings, while the program retains full authority over calculating the audio peaks and cutting the footage.

Main Mechanics / Sub Mechanics

- File Selection: The system provides an asset-loading interface where a user securely selects their video tracks (`.mp4`) and a music file (`.mp3` or `.wav`).
- Rhythm Tracking: The script feeds the music file into a processing algorithm that searches for rapid spikes in volume to map out the music's downbeats.
- Footage Slicing: The system loops through the unedited videos and slices them into smaller parts that fit perfectly within the time gaps between the music beats.

- Timeline Assembly: The program chains the edited video pieces back together chronologically, attaches the song underneath, and writes it out as a brand-new file.

AI Module(s)

- The tool features a lightweight peak-picking threshold model. Instead of searching for a single hardcoded volume level to identify a beat (which would completely fail if a song started out quiet and gets loud later during the chorus), the code dynamically calculates an ongoing average volume threshold for that exact part of the track. This ensures the program accurately catches drum hits during quiet intros as well as intense musical drops.
- Core Algorithm: Audio-Driven Video Synchronization Matrix. The following pseudo-code defines the programmatic logic used to ingest raw audio signals, calculate dynamic moving-average thresholds, map rhythmic peaks, and execute timeline cuts:

```
ALGORITHM: AudioBeatPacingOrchestrator INPUT: MusicFile (mp3/wav), RawVideoAssets
(Array of mp4), SpeedFlag (High/Low) OUTPUT: SynchronizedTimelineArray BEGIN // Step 1:
Ingest and flatten signal vector AudioSignal, SampleRate = Call librosa.load(MusicFile,
mono=True) // Step 2: Compute dynamic moving-average threshold window OnsetEnvelopes =
ComputeVolumeTransientSlopes(AudioSignal) DynamicThreshold =
CalculateMovingAverage(OnsetEnvelopes, window_size=100ms) // Step 3: Extract time-grid
coordinates where slopes cross threshold TimestampArray = [] FOR EACH Index, EnvelopeValue
IN OnsetEnvelopes DO IF EnvelopeValue > DynamicThreshold THEN Append (Index /
SampleRate) TO TimestampArray END IF END FOR // Step 4: Apply user pacing density filter
logic IF SpeedFlag == "Low" THEN TimestampArray =
FilterAlternatingElements(TimestampArray) END IF // Step 5: Timeline clipping loop
TimelineSegments = [] FOR i FROM 0 TO Length(TimestampArray) - 1 DO StartTime =
TimestampArray[i] EndTime = TimestampArray[i+1] VideoClip =
SelectAssetSource(RawVideoAssets) CutSegment = Call ffmpeg_extract_subclip(VideoClip,
StartTime, EndTime) Append CutSegment TO TimelineSegments END FOR // Step 6: Multi-track
composite compilation CompiledVideoTrack = ConcatenateTimeline(TimelineSegments)
FinalMuxedOutput = MixAudioVideoTracks(CompiledVideoTrack, MusicFile) RETURN
FinalMuxedOutput END
```

User Interface / Inputs / Sensors

- File Path Controls: Straightforward navigation buttons allowing the user to select local folders.
- Speed Settings: A choice parameter allowing the user to specify if the video should cut on every single beat (fast paced) or skip beats (slower pacing).

- Status Telemetry: Clear text printed directly to the screen telling the user what the program is doing in real time ("Analyzing Audio Tracks... 45%" or "Rendering Final Video").

Graphics / Animation / Video

- Target Codec: The software exports standard .mp4 video files using the widely compatible H.264 format, ensuring the finished clip plays properly on social media and phone apps.
- Resolution: The tool preserves the original 1080p high-definition layout and frame rates of the imported clips.

Audio

- Sound Standard: The engine loads standard 44.1kHz audio files. To save processing power, it temporarily flattens multi-channel stereo audio into a single channel to calculate the beats, then glues the full-quality stereo sound back onto the final video clip when rendering.

Production Documentation (Original COS550 Baseline)

Feature List (Original):

- F1 | Local Asset Ingestion System (Tools/Systems: Python 'os' module and built-in safe file I/O streams [3]) | System | High | Low
- F2 | Corrupted & Format File Validation (Tools/Systems: Native Python exception handlers to verify .mp4, .mp3, and .wav file headers [3]) | System | High | Medium
- F3 | Automated Audio Beat Analysis Core (Tools/Systems: 'Librosa' tracking engine computing volume-transient array jumps [1], [4]) | Tools | High | High
- F4 | Pacing Density Selection Modifier (Tools/Systems: Dynamic control parameters modifying clip frequency processing layers) | UI | Low | Low
- F5 | Synchronized Media Slicing Renderer (Tools/Systems: 'MoviePy' timeline engine utilizing timestamp scissor matrices [2]) | Tools | High | High
- F6 | Master Multi-Track Audio/Video Export (Tools/Systems: 'MoviePy' file compiler muxing dual stereo streams into an .mp4 container [2]) | System | High | Medium
- F7 | Live Console Progress Telemetry (Tools/Systems: Standard Python 'sys.stdout' stream printing live background execution percentages [3]) | UI | Low | Low

Master Task List & Development Roadmap (Original Milestones 1-3):

- Milestone 1:
 - T1 (F1, Dep: None): Run `python -m venv venv` and write dependency package versions to a `requirements.txt` configuration file according to standard Python packaging guidelines [3].
 - T2 (F1, Dep: T1): Parse local folder arguments using the native Python `os.path` module [3] to return absolute file paths from the computer disk.
 - T3 (F2, Dep: T2): Execute try-except exception catch blocks to evaluate and read binary header flags of `.mp4` and `.mp3` files as defined by standard file format specifications [3].
- Milestone 2:
 - T4 (F3, Dep: T3): Call the `librosa.load` function to ingest audio tracks and output a single-channel floating-point data array mapping raw signal voltage values [1].
 - T5 (F3, Dep: T4): Compute local volume transient shifts by executing a moving-average mathematical threshold loop across the audio array using Librosa's onset detection parameters [1], [4].
 - T6 (F3, Dep: T5): Export an array index list containing the exact float timestamps of every calculated rhythmic peak based on the digital signal processing grid [4].
 - T7 (F4, Dep: T6): Evaluate user choice configuration parameters via conditional statements to prune or retain alternating timestamp array elements.
- Milestone 3:
 - T8 (F5, Dep: T3, T6): Call `moviePy.video.io.ffmpeg_tools.ffmpeg_extract_subclip` [2] to programmatically split source video files at the exact timestamp array indices.
 - T9 (F5, Dep: T8): Concatenate the individual sliced video elements into a continuous timeline tracking array using MoviePy's sequence array compilation rules [2].
 - T10 (F6, Dep: T9): Combine the original background stereo audio track stream layer with the compiled chronological video timeline array layer via MoviePy multi-track composition hooks [2].
 - T11 (F6, Dep: T10): Call the `MoviePy.write_videofile()` system compiler [2] to render an H.264 encoded `.mp4` file container to the local disk workspace folder.

- T12 (F7, Dep: T5, T8): Print ongoing calculations to the system's standard output (stdout) stream [3] by dividing processed frame indices by total timeline frame counts.

Verification and Validation (V&V) (Original):

- T1 | Verification: The terminal screen displays a success message confirming virtual environment activation and a populated text file is written to requirements.txt per standard Python packaging guidelines [3]. | Validation: The developer possesses an isolated, sandboxed Python ecosystem containing all required external multimedia libraries.
- T2 | Verification: The console prints a structured string list containing the absolute system paths of every file discovered inside the targeted folder asset directories via the os.path module [3]. | Validation: The application successfully points to and locates raw editing assets saved anywhere on the local hard drive.
- T3 | Verification: The application runs without raising header format exception errors when reading the binary files of uncorrupted media assets as defined by format specifications [3]. | Validation: The video editor can load files safely without corrupt media breaking the application loop.
- T4 | Verification: An automated unit check verifies that a non-empty, single channel 1D floating-point data array is populated in the system memory via the librosa.load function [1]. | Validation: The background music is converted into a flat format that the volume-tracking core can read.
- T5 | Verification: The algorithm loop outputs mathematical transient data indexes whenever an audio amplitude slope crosses the configured threshold value using Librosa's onset parameters [1], [4]. | Validation: The program accurately traces changes in a musical rhythm to identify where the core downbeats drop.
- T6 | Verification: The program exports a sequential list of precise timeline float coordinates based on the digital signal processing grid [4]. | Validation: The system establishes an exact chronological blueprint layout grid for the video cuts to follow.
- T7 | Verification: Conditional execution gates prune or retain alternating timestamp elements depending on whether the user configuration flag is set to low or high pacing. | Validation: The editor can successfully choose whether the video cuts quickly or slowly by changing a single parameter setting.
- T8 | Verification: The external system calls to ffmpeg_extract_subclip [2] output separate, trimmed shorter video sub-clips saved directly into the temporary directory. |

Validation: Raw, unedited footage blocks are programmatically cut into smaller individual scenes at the exact rhythm markers.

- T9 | Verification: The compilation matrix combines the distinct visual segments into a single, continuous timeline tracking array sequence using MoviePy's sequence rules [2]. | Validation: The individual video clips flow consecutively in chronological order as a single unified sequence track.
- T10 | Verification: A timeline composition inspector verifies that the audio track layer and the video timeline layer are bound together via MoviePy multi-track composition hooks [2]. | Validation: The background music layer aligns and plays perfectly alongside the moving video.
- T11 | Verification: An H.264 encoded .mp4 file container is written directly to the local disk path using the .write_videofile() system compiler [2]. | Validation: The content creator receives a finalized, fully synchronized video file that plays smoothly in native desktop media players.
- T12 | Verification: The active console view window continually updates progress percentages from 0% to 100% directly to the sys.stdout stream [3]. | Validation: The user can easily monitor the status of long video rendering processes in real-time without the program appearing frozen.

Original References: [1] Librosa Development Team, "Librosa v0.10.1 Documentation: Audio and Time-Series Signal Processing," Librosa Core Modules, 2023. [Online]. Available: <https://librosa.org/doc/latest/> [2] Z. Zulko, "MoviePy v1.0.3: Video Editing Administration and Compositing Framework Suite," MoviePy Framework Documentation, 2020. [Online]. Available: <https://zulko.github.io/moviepy/> [3] Python Software Foundation, "The Python Standard Library Framework: v3.11.0 Core OS and System Output Documentation," Python Core Specifications, 2022. [Online]. Available: <https://docs.python.org/3/library/> [4] J. G. Proakis and D. G. Manolakis, "Onset Detection and Discrete Time Vector Processing," in Digital Signal Processing: Principles, Algorithms, and Applications, 4th ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2007, ch. 4, pp. 185-214.